



UNIVERSIDADE DE ÉVORA
ESCOLA DE CIÊNCIAS E TECNOLOGIA

DEPARTAMENTO DE INFORMÁTICA

Estágio de Mobile Development (Flutter) - DigitalWorks by Plexus Tech

Licenciatura em Engenharia Informática
Estágio-Projeto 2025/2026



Francisco Navarro Rodrigues

Orientador na Empresa: João Rouxinol

Orientador no Departamento: Professor Vitor Nogueira

Trabalho desenvolvido na empresa Digital Works By Plexus Tech, no âmbito da disciplina de Estágio-Projeto da Licenciatura em Eng. Informática.

Évora, 19 de junho de 2026

Tabela de conteúdos

Conteúdo

1	Introdução	3
1.1	Enquadramento institucional	4
1.2	Objetivos	4
1.3	Contribuições	5
1.4	Estrutura do documento	5
2	Ambiente de Trabalho	6
3	Fundamentação Teórica	7
3.1	Desenvolvimento Móvel com Flutter e Dart	7
3.2	Arquitetura de Software e Princípios de Design	7
3.3	Padrões de Gestão de Estado	8
3.4	Comunicação e Integração de Dados	8
3.5	Qualidade de Código e Testes	9
4	Contexto Operacional	10
4.1	Ambiente tecnológico	10
4.1.1	Hardware	10
4.1.2	Software	10
4.1.3	Ambiente aplicacional	11
4.2	Metodologia de trabalho	11
5	Trabalho desenvolvido	12
5.2	Introdução à <i>Framework</i> Flutter: Aplicação Base	13
5.2.1	Configuração e Lógica de Interface	13

5.2.2 Persistência de Dados (Cache)	14
5.2.3 Gestão de Estado e Injeção de Dependências	14
5.2.4 Arquitetura Limpa, Navegação 2.0 e Internacionalização	14
5.3 Desenvolvimento da Aplicação Móvel: Pokédex	15
5.3.1 Arquitetura e Injeção de Dependências	15
5.3.2 Camada de Domínio: Entidades e Use Cases	15
5.3.3 Camada de Dados e Gestão <i>Offline-First</i>	16
5.3.4 Camada de Apresentação e Navegação	17
5.3.5 Internacionalização (i18n) e Tematização Dinâmica	17
6 Avaliação crítica	18
Referências	19

1 Introdução

Este relatório foi concebido no âmbito da unidade curricular de Estágio-Projeto da Licenciatura em Engenharia Informática. O trabalho nele descrito foi realizado na empresa DigitalWorks by PlexusTech.

O desenvolvimento de aplicações móveis (*Mobile Development*) é caracterizado por diversos processos, tais como o desenho da arquitetura e das interfaces (UI/UX), a implementação (codificação), a fase de testes e, por fim, a disponibilização da aplicação. Durante o estágio na DigitalWorks, a aplicação móvel foi desenvolvida com recurso ao consumo de uma API, tendo sido utilizadas várias ferramentas e tecnologias fundamentais para a sua concretização.

As tecnologias de maior destaque neste projeto foram o Android Studio[3], o Flutter[1], a linguagem Dart[2], a arquitetura REST[4] e o Git[5]. O Android Studio consistiu no ambiente de desenvolvimento integrado (IDE) principal, sendo indispensável para a criação, edição de código e simulação de aplicações no ecossistema Android.

Por sua vez, o Flutter é uma *framework* de código aberto criada pela Google para o desenvolvimento híbrido (multiplataforma), permitindo criar aplicações compiladas nativamente a partir de um único código-fonte. Diretamente associada a esta ferramenta está a linguagem Dart, uma linguagem orientada a objetos e fortemente tipada, otimizada para o desenvolvimento rápido de interfaces através de recursos como o *Hot Reload*.

Para assegurar a comunicação e a integração de dados entre a aplicação móvel e os serviços externos, recorreu-se ao modelo arquitetural REST. Por fim, o Git desempenhou um papel crucial como sistema de controle de versões distribuído, sendo essencial para gerir o histórico de alterações e garantir a integridade do código ao longo de todo o ciclo de vida do desenvolvimento.

Este estágio proporcionou uma introdução prática ao desenvolvimento de aplicações móveis híbridas. Ao longo deste período, o plano de trabalhos focou-se não só na aprendizagem, mas também na aplicação rigorosa de princípios de engenharia de *software* de referência, tais como os princípios SOLID[7], bem como nas diretrizes fundamentais de Acessibilidade (*Accessibility*) [10] em contexto móvel.

O presente relatório encontra-se estruturado em várias secções. Estas descrevem detalhadamente as atividades desenvolvidas ao longo do estágio, enquadrando todo o desenvolvimento técnico e analítico efetuado durante este período.

1.1 Enquadramento institucional

No âmbito da unidade curricular de Estágio-Projeto, correspondente ao 3.º ano da Licenciatura em Engenharia Informática da Universidade de Évora, o estágio foi realizado na empresa DigitalWorks by PlexusTech, instalada no Parque do Alentejo de Ciência e Tecnologia (PACT), em Évora.

O plano de trabalhos decorreu num formato híbrido, conjugando o regime presencial com o teletrabalho, sendo a maioria dos dias remotamente. Durante os períodos presenciais, as atividades decorreram nas instalações da empresa, num ambiente de escritório partilhado com outros colaboradores e estagiários.

Apesar da partilha do espaço físico e do convívio no ambiente corporativo, o projeto que me foi proposto consistiu num trabalho individual. Desta forma, o desenvolvimento técnico e a execução das tarefas foram realizados de forma autónoma, não existindo uma integração direta numa equipa de desenvolvimento da empresa para a realização desta aplicação específica.

1.2 Objetivos

O estágio teve como ponto de partida uma fase inicial focada na aprendizagem e consolidação da ferramenta Flutter[1] e da linguagem Dart[2], através da realização de pequenos exercícios e aplicações exploratórias.

Após esta fase de adaptação, o objetivo principal consistiu no desenvolvimento de uma aplicação móvel, semelhante a uma Pokédex, capaz de consumir dados diretamente da PokéAPI[9]. Esta aplicação deve garantir diversas funcionalidades, tais como a listagem de Pokémon, pesquisa, filtragem por tipos, gestão de favoritos e a apresentação detalhada da linha evolutiva.

A nível técnico e de engenharia de software, o trabalho teve como objetivo a criação de uma solução robusta e escalável. Para isso, estabeleceu-se a meta de aplicar padrões como a *Clean Architecture*[6] e a gestão de estado através do padrão BLoC[8], garantindo também o cumprimento dos princípios SOLID[7] e de boas práticas de acessibilidade[10]. Outro objetivo técnico fundamental foi a implementação de testes unitários para assegurar a qualidade e estabilidade do código.

1.3 Contribuições

As principais contribuições deste projeto do ponto de vista técnico, a entrega de uma aplicação estruturada (utilizando *Clean Architecture* [6]) e com uma elevada cobertura de testes unitários resultou num código muito sólido. Este trabalho pode servir como referência interna de boas práticas e arquitetura para futuros desenvolvimentos em Flutter [1] dentro da empresa.

1.4 Estrutura do documento

O Capítulo 2 (Ambiente de Trabalho) descreve a organização onde decorreu o estágio — a DigitalWorks by PlexusTech —, abordando o enquadramento na estrutura da empresa, o regime de trabalho híbrido adotado e a fase de integração inicial (*onboarding*).

O Capítulo 3 (Fundamentação Teórica) expõe os conceitos teóricos e as metodologias que sustentam o trabalho prático desenvolvido. Justifica a adoção de tecnologias como a *framework* Flutter e a linguagem Dart, explorando em detalhe as abordagens arquiteturais utilizadas, como a *Clean Architecture*, os princípios SOLID e a gestão de estado BLoC.

O Capítulo 4 (Contexto Operacional) detalha o ambiente tecnológico diário, especificando o *hardware* e as ferramentas de *software* utilizados no desenvolvimento, emulação e controlo de versões, descrevendo também o enquadramento da aplicação isolada (*standalone*) e a metodologia de acompanhamento do projeto.

O Capítulo 5 (Trabalho Desenvolvido) documenta exaustivamente as fases práticas do estágio. Inicia-se com o relato da formação na gestão de repositórios (Git/GitHub) e da exploração inicial da *framework* (através de uma aplicação base), culminando com a descrição detalhada do desenvolvimento da aplicação móvel Pokédex, abordando a arquitetura de dados, lógicas reativas e *design* de interface.

Por fim, o Capítulo 6 (Avaliação Crítica) apresenta uma reflexão global sobre a experiência, abordando as dificuldades superadas, o aperfeiçoamento de competências técnicas de engenharia de *software* e a evolução a nível de competências não-técnicas (*soft skills*) em contexto profissional.

2 Ambiente de Trabalho

A DigitalWorks by PlexusTech é uma empresa tecnológica focada na criação de soluções digitais. O estágio decorreu no polo da empresa localizado no Parque do Alentejo de Ciência e Tecnologia (PACT), em Évora, onde assumi a função de Estagiário de Desenvolvimento Móvel (*Mobile Developer*) de flutter.

O foco principal do estágio foi a criação autónoma de uma aplicação móvel com recurso à *framework* Flutter. Apesar do projeto ser individual, o espaço de trabalho no escritório era partilhado com outros estagiários e empregados, o que proporcionou um ambiente colaborativo.

O processo de integração (*onboarding*) iniciou-se com a atribuição de equipamento informático e a realização de um módulo prático fornecido pela empresa. Este módulo teve como objetivo garantir a preparação do ambiente de desenvolvimento sobre sistemas de controle de versões, nomeadamente através da utilização do Git e do *software* SourceTree para a gestão de *branches*, resolução de conflitos e submissão de *Pull Requests* no GitHub.

A carga horária foi distribuída por dois dias na semana (quintas e sextas-feiras), das 09:00 às 18:00. A metodologia de trabalho adotada seguiu um formato híbrido: as sextas-feiras funcionaram sempre em regime remoto, enquanto que algumas quintas-feiras decorreram presencialmente no PACT.

O acompanhamento técnico e a gestão do projeto foram garantidos por um orientador interno da empresa, através de reuniões de ponto de situação todas as quintas-feiras de manhã, para apresentar o progresso da aplicação e validar as abordagens de desenvolvimento.

3 Fundamentação Teórica

Este capítulo apresenta os conceitos teóricos e as tecnologias que serviram de base técnica ao desenvolvimento do projeto de estágio.

Para contextualizar as decisões tomadas durante a construção da aplicação móvel, começa-se por introduzir a *framework* de desenvolvimento e a respetiva linguagem de programação adotada. Em seguida, exploram-se as abordagens estruturais do projeto, com destaque para a *Clean Architecture*, os princípios SOLID e o padrão de gestão de estado BLoC. Por fim, detalham-se os mecanismos de integração com serviços externos e as práticas de controle de qualidade e testes de *software*.

3.1 Desenvolvimento Móvel com Flutter e Dart

O Flutter é uma *framework* de código aberto, desenvolvida pela Google, focada na criação de aplicações multiplataforma (como Android e iOS) a partir de uma única base de código. A sua arquitetura é declarativa, o que significa que a interface gráfica é construída através de blocos visuais denominados *widgets*.

A linguagem de programação utilizada pelo Flutter é o Dart, uma linguagem orientada a objetos e otimizada para a criação de interfaces de utilizador. A principal vantagem técnica desta combinação reside nos seus métodos de compilação: utiliza *Just-In-Time* (JIT) durante o desenvolvimento, permitindo visualizar alterações de código instantaneamente (*Hot Reload*).

3.2 Arquitetura de Software e Princípios de Design

A *Clean Architecture* é um padrão de *design* de *software* que promove a separação de responsabilidades, dividindo o sistema em camadas independentes [6]. O seu principal objetivo é garantir que a lógica de negócio central não dependa de *frameworks* externas, bases de dados ou interfaces de utilizador. No contexto do desenvolvimento móvel, o projeto é tipicamente estruturado em três camadas principais:

- **Domain (Domínio):** É a camada mais interna e independente, que contém as entidades centrais e a lógica de negócio abstrata.
- **Data (Dados):** É responsável pela obtenção e persistência de informação, gerindo a comunicação com APIs externas ou bases de dados locais, e implementando os contratos (interfaces) definidos pela camada de domínio.

-
- **Presentation (Apresentação):** Foca-se exclusivamente na interface gráfica (UI) e na gestão de estado, exibindo a informação ao utilizador e reagindo às suas interações sem conhecer a origem dos dados.

Para complementar esta arquitetura estrutural, o desenvolvimento orienta-se pelos princípios SOLID, um conjunto de cinco regras fundamentais de programação orientada a objetos [7]. Estes princípios (Responsabilidade Única, Aberto/Fechado, Substituição de Liskov, Segregação de Interfaces e Inversão de Dependência), visam reduzir o acoplamento entre os diferentes componentes, resultando num código mais modular, tolerante a alterações e mais fácil de testar.

3.3 Padrões de Gestão de Estado

A gestão de estado é um elemento crucial no desenvolvimento de aplicações interativas, garantindo que a interface do utilizador reflita de forma consistente os dados e as interações mais recentes. Para este efeito, o padrão BLoC (*Business Logic Component*) é uma das abordagens mais consolidadas no ecossistema Flutter.

O padrão BLoC atua como um intermediário entre a camada de apresentação e a camada de dados, isolando completamente a lógica de negócio da interface gráfica. O seu funcionamento baseia-se num fluxo de dados reativo com recurso a *Streams*: a interface gráfica envia "Eventos" (disparados por ações do utilizador ou ciclos de vida da aplicação) para o BLoC, que processa a lógica correspondente e emite novos "Estados" em resposta. A interface, por sua vez, reage unicamente a estas alterações de estado, atualizando-se de forma automática.

Esta separação rigorosa garante um fluxo de dados previsível e unidirecional, o que facilita significativamente a manutenção do código, a adição de novas funcionalidades e, sobretudo, a execução de testes unitários isolados [12].

3.4 Comunicação e Integração de Dados

A integração com serviços externos é um requisito comum em aplicações móveis, sendo frequentemente realizada através do consumo de *Application Programming Interfaces* (APIs) baseadas no modelo arquitetural REST (*Representational State Transfer*) [4]. Esta arquitetura

permite a comunicação padronizada entre o sistema cliente (a aplicação móvel) e o servidor, tipicamente através de pedidos HTTP.

Os dados recebidos nestas comunicações são, na sua grande maioria, estruturados no formato JSON (*JavaScript Object Notation*), por ser um formato leve. Para que esta informação possa ser processada e apresentada de forma segura na aplicação, recorre-se ao processo de desserialização. Este processo consiste em extrair os dados brutos do formato JSON e mapeá-los para modelos de objetos nativos (neste caso, em Dart), garantindo que a aplicação trabalhe com dados estruturados.

3.5 Qualidade de Código e Testes

A garantia da qualidade do *software* é um processo focado em assegurar a robustez e a fiabilidade da aplicação. Neste contexto, os testes unitários desempenham um papel central, permitindo a validação isolada das parcelas mais pequenas de código, como as funções de lógica e a emissão de estados nos BLoCs [8].

A implementação regular de testes unitários ajuda a detetar erros numa fase precoce do desenvolvimento e cria uma salvaguarda contra regressões durante a introdução ou alteração de funcionalidades. Para avaliar a eficácia desta prática, utiliza-se a métrica de cobertura de código (*code coverage*), que indica a percentagem exata do código-fonte que é executada e validada pelos testes [12]. A conjugação de casos de teste bem estruturados com uma elevada cobertura é um dos principais indicadores de um projeto sustentável, e tolerante a falhas.

4 Contexto Operacional

Ao nível de hardware, utilizei um computador mac dado pela empresa para o desenvolvimento do projeto.

Ao nível de software, utilizei Flutter[1], Dart[2], Android Studio[3], Git[5], Github[13], SourceTree[14], Microsoft Teams[16], Fork[15].

A minha experiência com este software foi bastante positiva, embora já tivesse trabalho com alguns dos softwares descritos, não só em projetos da universidade, como também em projetos pessoais.

4.1 Ambiente tecnológico

4.1.1 Hardware

O equipamento informático fornecido pela empresa para a realização do projeto consistiu num computador Mac, dimensionado para suportar as exigências de compilação e desenvolvimento móvel multiplataforma.

Para a fase de testes e validação da interface da aplicação, recorreu-se a duas abordagens distintas. Numa primeira fase, utilizou-se a emulação virtual diretamente no computador. Numa segunda fase, para garantir o correto comportamento, e desempenho num ambiente real, os testes em dispositivos físicos foram conduzidos utilizando um telemóvel pessoal com sistema operativo Android, concretamente um Samsung Galaxy S22. A conjugação destes dois métodos garantiu uma perspetiva sobre o comportamento da aplicação.

4.1.2 Software

No que diz respeito às ferramentas de *software*, o ambiente de desenvolvimento integrado (IDE) foi o Android Studio, o qual foi conjugado com a ferramenta *Simulator* (nativa do sistema macOS) para a emulação e visualização da aplicação em tempo real no ecossistema iOS.

Para a gestão do código-fonte e controlo de versões alojado no GitHub, recorreu-se aos clientes gráficos *SourceTree* e *Fork*. Estas ferramentas facilitaram a visualização da árvore de

histórico e agilizam operações fundamentais de Git, tais como a criação de *branches*, a realização de *commits* e a gestão de *merges*.

No âmbito da colaboração corporativa, o *Microsoft Teams* assumiu o papel de ferramenta principal de comunicação.

4.1.3 Ambiente aplicativo

A aplicação desenvolvida opera num modelo *standalone* (isolado). Isto significa que o projeto não faz parte de um sistema maior da empresa, não estando integrado nos servidores privados, arquiteturas ou bases de dados internas da DigitalWorks.

O seu ambiente de operação interage de forma exclusiva com dois sistemas de dados: um serviço externo e um local. A nível externo, a aplicação comunica com um serviço público e aberto, a PokéAPI através de pedidos na rede para obter as informações e imagens necessárias. A nível local, interage com a memória do próprio dispositivo móvel (através de armazenamento local ou *cache*) para persistir dados gerados pelo utilizador, garantindo assim uma experiência fluida mesmo perante a ausência temporária de rede.

4.2 Metodologia de trabalho

Para a gestão e acompanhamento do projeto, adotou-se uma metodologia prática, flexível e baseada na comunicação direta. Dado tratar-se de um projeto de desenvolvimento individual, optou-se por não utilizar ferramentas complexas de gestão de tarefas ou *tickets*. Em vez disso, o planeamento e a revisão das atividades eram realizados de forma contínua e iterativa.

O alinhamento principal acontecia durante os dias de trabalho (a maior parte às quintas), através de reuniões de acompanhamento com o orientador de estágio. Nestes momentos, avaliava-se o progresso da aplicação, debatiam-se os desafios encontrados e definiam-se os próximos passos e objetivos.

5 Trabalho desenvolvido

O trabalho desenvolvido ao longo do estágio foi planeado de forma progressiva, dividindo-se numa fase de formação inicial e numa fase de desenvolvimento de *software* prático.

Na primeira semana de estágio, a prioridade foi a preparação e configuração do ambiente de trabalho. Através de documentação fornecida pela empresa, realizei uma formação focada no sistema de controle de versões (Git e GitHub), o que permitiu relembrar e consolidar os comandos essenciais a utilizar durante o projeto. Esta fase incluiu também uma breve apresentação das diversas ferramentas de *software* que viriam a ser adotadas.

Nas 4 semanas seguintes, o foco centrou-se na introdução à *framework* Flutter e à linguagem de programação Dart. Esta aprendizagem foi conduzida através da realização de fichas práticas, iniciando-se com a construção de uma aplicação base (uma *Counter App*), que serviu para compreender os conceitos fundamentais de *widgets* e gestão de estado simples.

A última e mais extensa tarefa ocupou os restantes meses do estágio e consistiu no projeto central: a criação de uma aplicação móvel estilo Pokédex. O objetivo do projeto foi consumir uma API pública de Pokémon (PokéAPI) para construir uma solução completa. O desenvolvimento foi guiado por uma lista de requisitos, dividida em funcionalidades obrigatórias e funcionalidades extra propostas pela empresa, havendo ainda total liberdade para a implementação de funcionalidades adicionais por iniciativa própria.

5.1 Formação Inicial: Sistema de Controlo de Versões (Git e Github)

No início do estágio, o primeiro passo prático consistiu na realização de um módulo introdutório focado na configuração do ambiente de trabalho e na consolidação de conhecimentos sobre Git e GitHub.

O processo iniciou-se com a criação de conta no GitHub e a instalação do *software* SourceTree, uma ferramenta gráfica utilizada para simplificar a gestão local e remota dos repositórios. A atividade prática central baseou-se num cenário simulado que exigiu a criação de um repositório privado focado na gestão de veículos.

Para garantir a correta aplicação das boas práticas de controle de versões, o exercício exigiu uma gestão rigorosa de ramificações (*branches*). Foi criada uma ramificação base de desenvolvimento (*develop*) e, posteriormente, uma ramificação chamada *epic* (*epic/add-vehicles*). A partir desta, o trabalho foi dividido em ramificações de

funcionalidades isoladas (*feature branches*) para a adição independente de dados e imagens referentes a carros, motos e barcos.

O ponto fulcral desta aprendizagem incidiu na integração de código (*merge*). Ao juntar as diversas *feature branches* na ramificação *epic*, foram gerados intencionalmente conflitos no ficheiro de texto principal. Este cenário obrigou à resolução manual dos conflitos, garantindo a preservação total dos dados introduzidos em todas as ramificações sem qualquer perda de informação. Após a integração e limpeza das ramificações temporárias, o módulo foi concluído com a submissão de um *Pull Request* para aprovação das alterações finais na ramificação *develop*, atribuindo a revisão do código aos orientadores da empresa.

5.2 Introdução à *Framework* Flutter: Aplicação Base

A segunda etapa do estágio centrou-se na aprendizagem da linguagem de programação orientada a objetos Dart e na exploração da *framework* móvel multiplataforma Flutter, que se rege pelo princípio fundamental de que "Tudo é um *Widget*".

5.2.1 Configuração e Lógica de Interface

O processo começou com a instalação e preparação do ambiente de desenvolvimento, incluindo o Flutter SDK, o Android Studio e os respetivos *plugins* de formatação e análise de código. Seguidamente, procedeu-se à manipulação prática de uma aplicação base focada na contagem numérica (*Counter App*). A nível de interface de utilizador, o trabalho consistiu em:

- Alterar a paleta de cores, reestruturar as pastas do projeto (separando componentes de interface) e adicionar novos elementos visuais e ícones de navegação.
- Implementar funcionalidades fundamentais, nomeadamente botões para incrementar, decrementar (com validação para impedir valores negativos) e repor a zeros o valor do contador.
- Introduzir interatividade avançada, tal como incremento através de um duplo toque no centro do ecrã e estilização condicional das cores dos números consoante o seu valor.
- Reforçar a interação e o *feedback* com o utilizador, incorporando barras de alerta (*SnackBar*) ativadas ao atingir objetivos numéricos, uma lista visual (*ListView*) com o

registo das últimas alterações e caixas de diálogo (*AlertDialog*) para a confirmação de operações irreversíveis.

5.2.2 Persistência de Dados (Cache)

Para dotar a aplicação com a capacidade de preservar a informação, mesmo após o seu encerramento, o projeto evoluiu para incorporar o pacote `shared_preferences`.

- Foi desenvolvida uma classe `StorageService` dedicada exclusivamente ao processamento de operações assíncronas para ler e gravar dados.
- As lógicas criadas possibilitaram a gravação em cache tanto do valor do contador, como do nome introduzido pelo utilizador através de um novo ecrã de definições.
- O título da aplicação foi adaptado para ser preenchido de forma dinâmica durante o arranque da mesma (`initState`), recuperando automaticamente da memória local os dados do utilizador.

5.2.3 Gestão de Estado e Injeção de Dependências

De forma a assegurar a robustez do código e separar a componente visual da lógica de negócio, a gestão imperativa feita nativamente através do método `setState` foi totalmente removida.

- Adotou-se o padrão arquitetural BLoC (*Business Logic Component*), criando ficheiros distintos para gerir os Estados da aplicação, os Eventos despoletados pelo utilizador e os respetivos *handlers* lógicos.
- A construção da interface gráfica tornou-se reativa, processando a reconstrução dos elementos visuais isoladamente por intermédio de componentes orientados a eventos, como o `BlocBuilder` e o `BlocListener`.
- Para centralizar a instanciação de serviços, configurou-se uma árvore de injeção de dependências com recurso à biblioteca `get_it`, registrando classes únicas como `LazySingleton` e classes partilhadas como `Factory`.
- Foi simultaneamente configurado um sistema estruturado de rotas nomeadas suportado na Navegação 1.0 para gerir a transição fluida entre os ecrãs de forma centralizada.

5.2.4 Arquitetura Limpa, Navegação 2.0 e Internacionalização

A última iteração da aplicação de contagem teve como intuito migrar todo o projeto para os padrões corporativos, através de uma Arquitetura Limpa (*Clean Architecture*) altamente escalável.

- O repositório transitou para um modelo dividido em camadas rigorosas: a camada de Domínio (contendo Exceções, Use Cases e *Interfaces* de repositórios), a camada de Dados (responsável por *Datasources* e pela execução real da persistência) e a camada de Apresentação.
- Para tratar potenciais cenários de falha na execução de código e acesso aos dados de forma estruturada, foi integrado o pacote *either_dart*, o qual permitiu devolver os resultados do código divididos em erros de operação e modelos de sucesso.
- Todo o sistema direcional foi reformulado para suportar a Navegação 2.0 com o pacote *go_router*, passando a utilizar um sistema declarativo baseado no estado e suportando o encadeamento hierárquico de rotas.
- Como ponto final, preparou-se toda a solução para suportar nativamente múltiplos idiomas (internacionalização), processando a extração dos textos diretamente de ficheiros autogerados de tradução.

5.3 Desenvolvimento da Aplicação Móvel: Pokédex

O projeto central e de maior exigência técnica do estágio consistiu no desenvolvimento de uma aplicação móvel interativa desenvolvida em Flutter, com o objetivo de apresentar dados da *PokéAPI*. O projeto foi estruturado seguindo os princípios da *Clean Architecture*, garantindo a separação de responsabilidades, testabilidade e escalabilidade através da divisão em três camadas principais: Domínio, Dados e Apresentação.

5.3.1 Arquitetura e Injeção de Dependências

Para garantir o desacoplamento das diversas camadas, implementou-se o padrão de *Service Locator* com recurso ao pacote *get_it*. A inicialização das dependências foi centralizada num método global *initInjector*, que regista instâncias partilhadas, como o *http.Client* para os pedidos de rede e o *SharedPreferences* para o armazenamento local. O registo foi modularizado por camadas, recorrendo aos ficheiros *domain_di.dart*, *data_di.dart* e *presentation_di.dart*, o que permitiu injetar automaticamente os repositórios nos *Use Cases* e, por sua vez, injetar estes nos respectivos blocos lógicos da interface (BLoCs).

5.3.2 Camada de Domínio: Entidades e Use Cases

A camada de domínio, sendo o núcleo independente da aplicação, definiu a entidade Pokémon, responsável por encapsular os atributos centrais como identificador, nome, imagem, tipos e estatísticas. Esta entidade estende a classe `Equatable` para facilitar comparações baseadas em valores.

Foi estabelecido o contrato abstrato `PokemonRepository`, definindo as operações disponíveis sem revelar a sua implementação técnica. A lógica de negócio foi isolada em *Use Cases* dedicados a ações únicas e específicas, tais como `GetPokemonsUseCase`, `GetPokemonDetailsUseCase` e `ToggleFavoriteUseCase`.

Para um tratamento de erros robusto e previsível, adotou-se o paradigma da programação funcional através do pacote `fpdart`. Todos os *Use Cases* e métodos do repositório retornam uma classe `Either`. Este padrão obriga a que qualquer chamada trate, em tempo de compilação, o cenário de falha (lado Left, contendo uma classe padronizada `AppError`) e o cenário de sucesso (lado Right, com o retorno dos dados esperados), eliminando o uso imprevisível de exceções não capturadas.

5.3.3 Camada de Dados e Gestão *Offline-First*

A camada de dados ficou encarregue de fornecer informações ao domínio. O modelo `PokemonModel` estende a entidade e implementa a lógica de serialização JSON (`fromJson` e `toJson`), lidando com as peculiaridades e conversões necessárias dos dados em bruto oriundos da API.

A recolha de dados foi dividida em duas fontes distintas (*Data Sources*):

- **Remota (`PokemonRemoteDataSource`):** Recorre à biblioteca `http` para efetuar chamadas *RESTful* aos vários terminais da *PokéAPI*, processando as listagens, detalhes, tipologias e percorrendo a complexa árvore iterativa da cadeia evolutiva de cada Pokémon.
- **Local (`PokemonLocalDataSource`):** Recorre ao `SharedPreferences` para persistir dados localmente no dispositivo. Para além de guardar a lista de identificadores dos Pokémon favoritos do utilizador, guarda uma cópia em cache (texto em formato JSON) das listas já carregadas e dos detalhes de cada pokémon visitada.

Esta arquitetura revelou o seu poder na implementação do repositório unificado (`PokemonRepositoryImpl`). Este repositório coordena as duas fontes de dados: sempre que é efetuado um pedido remoto e este tem sucesso, os dados são atualizados na cache local.

Em caso de falha de rede ou interrupção de serviço da API, o bloco de código transita para um bloco `catch`, que vai recuperar os dados guardados no armazenamento local. Isto proporciona ao utilizador uma experiência *offline-first*, permitindo visualizar a Pokédex e aceder aos favoritos mesmo sem ligação à Internet.

5.3.4 Camada de Apresentação e Navegação

A experiência visual foi construída de forma reativa sob o padrão BLoC (*Business Logic Component*). A aplicação foi dividida em núcleos geridos por diferentes gestores de estado: `PokemonBloc` (listagem principal, filtros e paginação infinita), `PokemonDetailsBloc` (detalhes e cadeia evolutiva) e `FavoritesBloc` (lista local de guardados).

A navegação declarativa foi implementada através do `go_router`, onde o ficheiro estrutural `Routes` configura animações de transição personalizadas entre os ecrãs: transições graduais de opacidade (`FadeTransition`) para os detalhes e entradas laterais deslizantes (`SlideTransition`) para os ecrãs de favoritos, oferecendo fluidez e polimento ao sistema.

A nível de Interface do Utilizador (UI) e de Experiência de Utilizador (UX), incorporaram-se funcionalidades avançadas, tais como:

- Otimização do tempo de carregamento de ecrã com contentores esqueleto cintilantes do `skeletonizer` antes da resolução dos estados BLoC.
- Animações ricas e interativas utilizando vetores JSON via `Lottie` (e.g., quando uma lista retorna vazia ou ao favoritar um pokémon).

5.3.5 Internacionalização (i18n) e Tematização Dinâmica

Para conferir flexibilidade e respeitar as boas práticas globais, o ponto de entrada da aplicação, no widget global `MyApp`, injeta as configurações de localização e tematização. A biblioteca base de suportes locais do Flutter interage com as delegações do `AppLocalizations`, suportando oficialmente três idiomas (Português, Inglês e Espanhol).

Em termos de aparência visual, o sistema incorpora um `ThemeProvider` assente no estado base do `Provider`, injetando diretamente as definições globais do esquema de cores predefinido `themeData` em toda a aplicação. Isto permitiu a criação de um tema dinâmico e flexível de *Light Mode* e *Dark Mode*, enriquecendo a experiência do utilizador final.

6 Avaliação crítica

A realização deste estágio na DigitalWorks by PlexusTech revelou-se uma etapa fundamental no meu percurso académico e profissional, proporcionando uma ponte sólida entre os conhecimentos teóricos adquiridos na universidade e a realidade. Fiquei surpreendido, pela positiva, com a diferença entre desenvolver um projeto académico e conceber um produto com padrões de nível empresarial.

Do ponto de vista técnico, a experiência foi extremamente enriquecedora. A curva de aprendizagem inicial com o Flutter e a linguagem Dart foi desafiante, mas permitiu-me compreender o poder do desenvolvimento de interfaces declarativas e multiplataforma. O maior ensinamento técnico que retiro deste estágio foi a percepção de que escrever código apenas para que o mesmo seja "funcional" não é suficiente. A adoção rigorosa de metodologias como a *Clean Architecture* e o padrão BLoC ensinou-me a importância da separação de responsabilidades, resultando num código mais limpo e tolerante a falhas. Além disso, a gestão de dependências, o tratamento correto de dados assíncronos e a integração com APIs reais mostraram-me a verdadeira complexidade de criar uma aplicação móvel resiliente.

No que diz respeito à vertente não-técnica (*soft skills*), o estágio foi igualmente transformador. Sendo o desenvolvimento da Pokédex um projeto individual, fui obrigado a ter um nível de autonomia, responsabilidade e gestão de tempo. Aprendi a planear as minhas próprias tarefas, a procurar soluções de forma independente quando bloqueado e a gerir a frustração à resolução de problemas complexos de *software*.

Apesar da natureza individual do projeto, a comunicação desempenhou um papel crucial. As reuniões de acompanhamento (*status meetings*) realizadas às quintas-feiras com o meu orientador foram essenciais, estes momentos obrigaram-me a treinar a minha capacidade de comunicação técnica, ensinando-me a reportar o progresso de forma clara, a expor dúvidas e a justificar de forma fundamentada as escolhas de arquitetura que tomava no código. O facto de partilhar o ambiente de escritório presencial com outros estagiários também contribuiu para um espírito de entreajuda e partilha de ideias, o que tornou a experiência menos solitária e muito mais colaborativa.

Em suma, o balanço desta experiência é extremamente positivo. Sinto que consolidei os meus objetivos propostos e termino este estágio com uma visão muito mais madura e estruturada sentindo-me amplamente mais preparado e motivado para os futuros desafios.

Referências

- [1] Google. (2024). *Flutter - Build apps for any screen*. Disponível em: <https://flutter.dev>. Consultado em junho de 2024.
- [2] Google. (2024). *Dart programming language*. Disponível em: <https://dart.dev>. Consultado em junho de 2024.
- [3] Google. (2024). *Android Studio*. Disponível em: <https://developer.android.com/studio>. Consultado em junho de 2024.
- [4] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Ph.D. dissertation, University of California, Irvine.
- [5] Torvalds, L., et al. (2024). *Git*. Disponível em: <https://git-scm.com>. Consultado em junho de 2024.
- [6] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [7] Martin, R. C. (2000). *Design Principles and Design Patterns*. Object Mentor.
- [8] Angelov, F. (2024). *Bloc Library - A predictable state management library for Dart*. Disponível em: <https://bloclibrary.dev>.
- [9] PokéAPI. (2024). *PokéAPI - The RESTful Pokémon API*. Disponível em: <https://pokeapi.co>.
- [10] W3C. (2015). *Mobile Accessibility: How WCAG 2.0 and Other W3C/WAI Guidelines Apply to Mobile*. Disponível em: <https://www.w3.org/TR/mobile-accessibility-mapping/>.
- [11] ECMA International. (2017). *The JSON Data Interchange Syntax*. Standard ECMA-404.
- [12] Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.
- [13] GitHub, Inc. (2024). *GitHub*. Disponível em: <https://github.com>.
- [14] Atlassian. (2024). *Sourcetree - A free Git client for Windows and Mac*. Disponível em: <https://www.sourcetreeapp.com>.

[15] Danilov, D., & Danilova, T. (2024). *Fork - a fast and friendly git client for Mac and Windows*. Disponível em: <https://git-fork.com>.

[16] Microsoft. (2024). *Microsoft Teams*. Disponível em: <https://www.microsoft.com/pt-pt/microsoft-teams>.